

One room, many peers

The win: you can explain, to an interviewer or to yourself, how several people edit one document at once and all end up with the same one, and you can point at the line of code for every claim.

How to use this page. Every code block is cut from the repo at commit 94222ab, so what you read is what runs. It is long: one section a day is a fine pace. Sections 3, 5 and 9 are the ones an architect interview will dig into.

```
./init.sh                                # Postgres + api + sync
pnpm --filter @noon/web dev              # the canvas, http://localhost:5173
make sim                                # the reconcile simulator, every committed
seed
node apps/sync/src/sim-cli.ts --seed 7 --trace | head -40
```

Try it: open <http://localhost:5173> in two windows, press *New document* in one, paste its address into the other, and edit in both.

The map: one edit, from a click to everyone's screen

- 1 · A closed set of messages: the discriminated union
- 2 · One planner, two appliers
- 3 · The event loop is your lock, until you write `await`
- 4 · The room: eight checks, in an order that matters
- 5 · Exactly once, over a connection that lies
- 6 · A pure core in a thin shell, and dependencies as parameters
- 7 · WebSockets: what HTTP did for you, and now you do
- 8 · Optimistic editing: two documents and one invariant
- 9 · Rate limiting: where you charge matters more than how
- 10 · Presence: the message that is not an op
- 11 · React meets a store that is not React's
- 12 · Types as data: reading TypeScript with TypeScript
- 13 · How you test concurrency without sleeping
- 14 · What the type system could not see, epic 2 edition

Your turn: three drills

Recall, then test yourself

The map: one edit, from a click to everyone's screen

Epic 2 is the heart of the product: several people (and later an AI and a git peer) edit one document at once, and everyone ends up with the same document. The design is the one Figma uses: **the server decides the order**. There is no CRDT and no merge function; there is one room per document, and the room hands out sequence numbers.

STEP	WHERE	WHAT HAPPENS
1	apps/web/src/Canvas.tsx	a click builds an op (one of four) and calls <code>peer.submit(op)</code>
2	packages/peer-client/src/replica.ts	<code>local()</code> : checked against what the user SEES, applied at once (optimistic), queued as pending
3	packages/peer-client/src/peer.ts	<code>flush()</code> : written to the WebSocket, at most 50 unanswered at a time
4	apps/sync/src/server.ts	frame size cap, JSON, contract check; presence goes one way, ops the other
5	apps/sync/src/room.ts	budget, dedupe, stale?, validate, limits, persist, apply, take the seq, remember, broadcast
6	packages/peer-client/src/replica.ts	<code>receive()</code> : the sender's copy is its acknowledgement; everyone else's is a remote edit
7	apps/web/src/usePeer.ts	<code>revision</code> moved, so React renders

Two ideas carry the whole epic. **First: a pure core in a thin shell.** `room.ts` has no socket in it and `replica.ts` has no socket in it; `server.ts` and `peer.ts` are only wires. That is why the rules can be tested in microseconds, and why a simulator can drive the real code. **Second: every message is a value of a closed set of shapes**, checked at both ends by the same schema.

one room per document, in memory, single-threaded

an actor (Akka), or a `synchronized` aggregate

an async task owning its state

ops + sequence numbers; last writer wins per property

event sourcing with a single writer

the same

optimistic client + rollback

rare on the JVM server side; common in Android/iOS clients

rare

1 • A closed set of messages: the discriminated union

An op is *one of four things*. In Java you would write a `sealed interface Op` permits `AddNode`, `MoveNode`, ... with four records. TypeScript has no classes here at all: an op is a plain object, and the field `type` tells the compiler which of the four shapes it is.

FROM `packages/contracts/src/index.ts`

```
export const Op = z.discriminatedUnion("type", [
  z.strictObject({ type: z.literal("add_node"), nodeId: NodeId, parentId: NodeId, index:
    z.number().int(), component: z.string().min(1).max(100), props: Props }),
  // `index` is the node's FINAL position among the new parent's children (SPEC §2.4).
  z.strictObject({ type: z.literal("move_node"), nodeId: NodeId, newParentId: NodeId,
    index: z.number().int() }),
  z.strictObject({ type: z.literal("remove_node"), nodeId: NodeId }),
  // value null = remove the prop, so the component's own default applies again.
  z.strictObject({ type: z.literal("set_prop"), nodeId: NodeId, key: PropKey, value:
    PropValue.nullable() }),
]);
export type Op = z.infer<typeof Op>;
```

Now watch what the compiler does with it. Inside `case "move_node":` the variable `op` has a `newParentId`; inside `case "set_prop":` it does not, and using it is a compile error. This is called **narrowing**: a runtime check on the discriminant changes the static type in that branch.

FROM `packages/doc-model/src/index.ts`

```
case "remove_node": {
  const node = nodeOf(doc, op.nodeId);
  if (!node || node.parentId === null) return undefined; // unknown node, or the root
  const parent = nodeOf(doc, node.parentId);
  if (!parent) return undefined;
  return { set: [{ ...parent, children: parent.children.filter((id) => id !==
    node.id) }], remove: subtree(doc, node.id) };
}
case "set_prop": {
  const node = nodeOf(doc, op.nodeId);
  if (!node || node.parentId === null) return undefined; // unknown node, or the root
```

```
(which has no props)
    const current = Object.hasOwn(node.props, op.key) ? node.props[op.key] : undefined;
    if (op.value === null ? current === undefined : Object.is(current, op.value))
return undefined;
    const props = Object.fromEntries(Object.entries(node.props).filter(([key]) => key
!== op.key));
    if (op.value !== null) props[op.key] = op.value;
    return { set: [{ ...node, props }], remove: [] };
}
```

FROM `packages/doc-model/src/index.ts`

```
default: {
    const unreachable: never = op; // a fifth op type stops compiling here (Lesson 0,
section 3)
    return unreachable;
}
}
```

The `never` trick. After four `case` s, nothing is left, so in `default` the type of `op` is `never` (the empty type). Assigning it to a `never` variable compiles *only while the switch is complete*. Add a fifth `op` to the contract and this line stops the build, in every switch that uses the trick. Java 21 gives you the same through pattern-matching `switch` over a sealed type; Python's `match` does not check exhaustiveness at all (mypy can, with `assert_never`).

Three tools for working with unions that you will see all over this epic:

- `Extract<ServerMessage, { type: "op" }>` : "the member of the union whose `type` is `"op"`". A type-level filter. `Exclude` is its opposite (`replica.ts` uses it to say "every server message except the presence ones").
- `{ ... }` satisfies `ClientMessage` : checks a literal against a type *without widening it*. `peer.ts` uses it on the frame it is about to send: a typo in a field name is a compile error, right where the frame is built.
- `z.strictObject` versus `z.object` : a *client* message is strict (an unknown key is an error: a client must not be able to slip in a `name`); a *server* message is loose (unknown keys are dropped: an old tab must survive a newer server).

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
union of object types + a literal discriminant	sealed interface + records	Union[...] + Literal discriminator (pydantic)
narrowing by switch (op.type)	pattern-matching switch	match / isinstance
exhaustiveness: never	compiler error on a missing case (sealed)	assert_never + mypy only
no classes, no instanceof : data crosses the wire as JSON and comes back as the same type	needs Jackson polymorphic config	pydantic discriminated unions

CASE STUDY · FOUND BY REVIEW OR BY A TEST THAT FLAKED

The forward-compatibility rule had a hole: new VALUES inside a known message type.

peer.ts skips a message whose type it does not know, so an old tab survives a server that learned a new message. Then the rate limit added a new *reject reason*, `rate_limited`. An old tab knows the type `"rejected"`, parses it, fails on the unknown reason, and that was fatal: the session ended and every unsent edit was dropped. The fix: a refusal with an unknown reason means "not applied", which is all a client needs to know. *A closed union is a promise to the compiler, not to last year's client.*

2 · One planner, two appliers

What does an op *do*? That decision is made in exactly one function, `plan()`, which reads the document and returns a patch (or `undefined` for "changes nothing"). Two tiny appliers turn the patch into a result:

FROM `packages/doc-model/src/index.ts`

```
export function applyOp(doc: Doc, op: Op): Doc {
  const patch = plan(doc, op);
  if (!patch) return doc;
  const doomed = new Set(patch.remove);
  const nodes: Doc["nodes"] = {};
  for (const [id, node] of Object.entries(doc.nodes)) if (!doomed.has(id)) nodes[id] = node;
  for (const node of patch.set) nodes[node.id] = node;
  return { ...doc, nodes };
}

/**
```

```

* IN PLACE: edits `doc` and says whether anything changed. Same decisions as applyOp
(both ask
* plan()), but O(what changed) instead of O(nodes). The room and journal replay use this
one:
* replaying 10,000 ops through the copying applier took 13-26 s, against a 2 s budget
(F19).
*/
export function applyOpInto(doc: Doc, op: Op): boolean {
  const patch = plan(doc, op);
  if (!patch) return false;
  // eslint-disable-next-line @typescript-eslint/no-dynamic-delete -- ids are data; they
are hasOwn-checked and never reserved names
  for (const id of patch.remove) delete doc.nodes[id];
  for (const node of patch.set) doc.nodes[node.id] = node;
  return true;
}

```

Why two? The browser needs a *pure* function: it must be able to throw a guess away. The server needs speed: replaying 10,000 ops through the copying version took 13 to 26 seconds (it copies the node map every time, so replay is $O(n^2)$); the in-place version does it in a fraction of a second. Writing the rules twice would have been the classic bug farm, so both ask the same planner. A property test runs them side by side on random ops and demands identical documents.

Structural sharing. `applyOp` builds a new node map but reuses every node object it did not touch. "Immutable" in JavaScript never means deep copies; it means *new containers, shared contents, and a promise not to edit the contents*. The replica leans on that promise in section 8.

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
<code>{ ...parent, children: [...] }</code> spread = shallow copy	records + <code>withX()</code> / builders	<code>dataclasses.replace</code>
nothing stops mutation; the discipline is tested (frozen nodes in a property test)	<code>final</code> fields, immutable collections	<code>frozen=True</code>
plain objects as maps: <code>doc.nodes[id]</code>	<code>HashMap</code>	<code>dict</code>

CASE STUDY · FOUND BY REVIEW OR BY A TEST THAT FLAKED

A node named `__proto__` crashed the server; one named `constructor` corrupted the document.

This one is pure JavaScript, and nothing in Java or Python prepares you for it. A plain object used as a map *inherits* from `Object.prototype`. So `doc.nodes["constructor"]` is not undefined: it is the `Object` function. And assigning to `doc.nodes["__proto__"]` does not store a key, it calls a setter. Two defences, both needed: the contract refuses such names (`notOnObjectPrototype`), and every lookup goes through `Object.hasOwn`:

FROM `packages/doc-model/src/index.ts`

```
export const nodeOf = (doc: Doc, id: string): DocNode | undefined =>
  (Object.hasOwn(doc.nodes, id) ? doc.nodes[id] : undefined);
```

A `Map` has none of these problems, but a `Map` does not survive `JSON.stringify`, and this document lives on the wire and in a `jsonb` column. That is the trade.

3 · The event loop is your lock, until you write `await`

The room has no mutex, no `synchronized`, no atomic counter, and it is correct. Node runs your JavaScript on **one thread**, and a function, once started, runs to its end (or to its next `await`) before anything else runs. Two WebSocket frames cannot be "in the room at the same time".

The catch is in the parentheses. An `await` is a point where other code runs. The room awaits only one thing, the journal (`journal.append` and its lookups, from epic 6). Without care, op B would be validated against a document that op A has not changed yet. So ops go through a queue made of one promise:

FROM `apps/sync/src/room.ts`

```
/** Resolves when this op has been fully handled (accepted and broadcast, or
refused). Never rejects. */
submit(peer: Peer, clientOp: ClientOp): Promise<void> {
  // The budget is charged HERE, as the op arrives, not when the queue reaches it:
  behind a slow
  // journal the bucket would refill while ops wait, nothing would ever be refused,
  and a flood
  // would simply pile up in memory. Every submission costs the same, whatever it
  turns out to be.
  if (!peers.has(peer) || overBudget(peer, clientOp.opId)) return tail;
  const done = tail.then(() => handle(peer, clientOp));
  tail = done.catch(() => undefined); // one failure must not jam the queue for every
  later op
```

```
    return tail;
  },
```

`tail` is "the promise of everything submitted so far". Each new op chains onto it and becomes the new tail. That is a complete, fair, in-order work queue in three lines, with no library. The `.catch` matters: a rejected promise would otherwise poison every later `.then`.

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
single thread; interleaving only at <code>await</code>	real threads; interleaving anywhere	asyncio: same as Node; threads: GIL, interleaving anywhere
queue = a promise chain	<code>ExecutorService</code> with one thread, or an actor mailbox	<code>asyncio.Queue</code> + one worker task
CPU-bound work blocks everyone	use more threads	same problem as Node

CASE STUDY · FOUND BY REVIEW OR BY A TEST THAT FLAKED

A test that failed one run in six was a real lost edit.

The server attached its `message` listener *after* `await` ing the document load. A frame that arrived during the load was emitted to nobody and lost. In Java the socket would have been read by your own loop, so nothing could arrive "early". In Node, events fire whenever the loop is free: **attach listeners before the first `await`**, and buffer. The fix queues early frames and replays them after the welcome. The lesson beyond Node: a flaky test is a bug report with a bad title.

Principal-level point: single-threaded does not mean free of races. It means races can only happen *across* awaits, which makes them fewer and findable. Read every `await` in shared-state code as "anything can have changed when I come back".

4 · The room: eight checks, in an order that matters

FROM `apps/sync/src/room.ts`

```
async function handle(peer: Peer, { opId, baseSeq, op }: ClientOp): Promise<void> {
  const refuse = (reason: Extract<ServerMessage, { type: "rejected" }>["reason"]): void => {
    if (reason === "unavailable") storageFailed();
    peer.send({ type: "rejected", opId, reason });
  };

  if (!peers.has(peer)) return; // dropped while this op waited in the queue
```

```

    // F24: asked when the op's TURN comes, not when it arrived: an op queued before a
    demotion is refused too.
    if (!peer.mayEdit) {
        refuse("forbidden");
        return;
    }
    // Before the dedupe: even an answer from memory is an acknowledgement, and a read-
    only room gives none.
    if (readOnly) {
        refuse("unavailable");
        return;
    }

    // 1. Dedupe FIRST. A client that never saw its acknowledgement sends the op again
    and must get
    // the original answer; validating first would turn an ordinary retry into
    "duplicate_node".
    const key = `${peer.actor.id}:${opId}`;
    const before = remembered.get(key);
    if (before) {
        peer.send({ type: "op", ...before.op });
        return;
    }
    if (rememberedNoOps.has(key)) {
        peer.send({ type: "ack", opId });
        return;
    }
    // An op written before the oldest thing the room remembers MAY already have been
    applied. The
    // journal can tell: found, it gets its original answer; not found, it never was, and
    is judged now.
    // Without a journal the room cannot tell, and applying it twice would move a node
    twice: resync.
    // A client that forged a high baseSeq (buggy or lying) skips that check, so every
    refusal below,
    // and the no-op ack, asks the journal first too: a resend gets its first answer,
    whatever it claims to
    // have seen (noon-mo3.1.1). Asked at most once per op, and only on those rare paths.
    let asked = false;
    /** True when the op has been answered: with its original row, or "unavailable". */
    const answeredFromJournal = async (): Promise<boolean> => {
        if (!journal || asked) return false;
        asked = true;
        let original;
        try {
            original = await journal.find(peer.actor.id, opId);
        } catch {
            refuse("unavailable");
            return true;
        }
        if (original) peer.send({ type: "op", ...original });
        return original !== undefined;
    };
    if (baseSeq < forgottenUpTo) {
        if (!journal) {

```

```

        refuse("stale");
        return;
    }
    if (await answeredFromJournal()) return;
}

// 2. The document's rules first (they give the precise reason: a move into its own
subtree is a
//     "cycle", not a depth problem), then the room's limits. A refusal goes to the
sender only.
const verdict = validate(doc, op, manifest);
if (!verdict.ok) {
    if (!(await answeredFromJournal())) refuse(verdict.reason);
    return;
}
if (overLimit(op)) {
    if (!(await answeredFromJournal())) refuse("document_limit");
    return;
}
// Keystone 4: validate() only sees the document as it is; an id that was removed is
in the journal.
// ponytail: one indexed read per add (adds are rare next to drags); without a
journal, unchecked.
if (op.type === "add_node" && journal) {
    let reused;
    try {
        reused = await journal.everAdded(op.nodeId);
    } catch {
        refuse("unavailable");
        return;
    }
    if (reused) {
        if (!(await answeredFromJournal())) refuse("duplicate_node");
        return;
    }
}

// An op that changes nothing (the value is already that) is answered and goes no
further: no seq,
// no journal row, no broadcast. Remembered like any other, so that its resend is
still a no-op.
if (!changes(doc, op)) {
    if (await answeredFromJournal()) return;
    rememberedNoOps.add(key);
    if (rememberedNoOps.size > REMEMBERED_NO_OPS)
rememberedNoOps.delete(rememberedNoOps.values().next().value ?? key);
    peer.send({ type: "ack", opId });
    return;
}

// 3. Durable first, then applied, then announced. The number is only TAKEN once the
op is durable,
//     so a failed write leaves no gap. The ACTOR comes from the verified session
(SPEC §2.3).
const sequenced: SequencedOp = { seq: seq + 1, opId, actor: peer.actor, op };

```

```

let original;
try {
  original = await journal?.append(sequenced);
} catch {
  refuse("unavailable");
  return;
}
// A resend the room had forgotten, with a baseSeq that got it past the check above
(a buggy or lying
// client). The journal's unique key caught it: the original answer, and nothing is
applied twice.
if (original) {
  peer.send({ type: "op", ...original });
  return;
}
accept(sequenced);
}

```

Each step's position was earned by a bug:

- **Dedupe before validate.** A client that never saw its acknowledgement sends the op again. Validate first, and an honest retry of `add_node` is answered `duplicate_node`.
- **Stale before validate.** If the op is older than anything the room remembers, the room cannot know whether it applied it. It says so (`stale`) instead of guessing.
- **The document's rules before the room's limits**, because they give the precise reason (a move into your own subtree is a *cycle*, not a depth problem).
- **Durable, then applied, then announced.** The sequence number is only *taken* once the op is durable, so a failed write leaves no gap, and nobody ever hears of an op that could be lost.
- **The actor is stamped here**, from the verified session. A client cannot claim "the AI did this".

And the budget is not in this function at all: it is charged in `submit()`, as the op *arrives*. Section 9 says why.

CASE STUDY · FOUND BY REVIEW OR BY A TEST THAT FLAKED

A graceful shutdown lost every unsaved edit.

`close()` terminated the sockets and then awaited the pending saves. There were none: the `close` handlers that *start* the saves run on a later tick of the event loop. The function returned, the process exited. The fix is to save every room first, then close. In Node, "I called it" and "it has happened" are different moments; when order matters, `await` the thing itself.

5 · Exactly once, over a connection that lies

A network gives you two honest choices: at most once, or at least once. "Exactly once" is built on top: **send at least once, and make the receiver recognise repeats**. Every op carries a random `opId`; the room remembers what each `(sender, opId)` became and answers a repeat with the original answer.

The memory is bounded (20,000 ops), so there must be a rule for what falls outside it. That is `baseSeq`: the last sequence number the client had seen when it *wrote* the op. Older than anything remembered? `stale`.

FROM `packages/peer-client/src/replica.ts`

```
if (reason === RejectReason.enum.stale) {
  // The room no longer remembers whether it applied this op. If the op would change
  nothing, it
  // is already there (or no longer makes sense): drop it quietly. If it WOULD change
  something,
  // that proves nothing: it may have been applied and then overwritten, and sending
  it again would
  // put an old write on top of a newer one, or bring back a node someone removed. So
  only an op
  // that no EARLIER connection ever carried is sent again; any other is given up and
  the user is told.
  // E6.1a's journal remembers every opId for good; from then on a resend is always
  safe.
  const alreadyThere = applyOp(confirmed, mine.op) === confirmed;
  if (!alreadyThere && !mine.maybeApplied && mine.staleCount < MAX_STALE_RETRIES) {
    mine.staleCount++;
    mine.baseSeq = seq; // honest again: it is now written against what we have seen
    // It goes out again, and so does everything after it: our own edits must reach
    the room in the
    // order we made them (red, then blue). The room answers the repeats from its
    memory.
    for (const each of pending.slice(pending.indexOf(mine))) each.inFlight = false;
    return effects;
  }
  pending = pending.filter((each) => each !== mine);
  rebuild();
  effects.settled.push({ opId, outcome: alreadyThere ? { ok: true } : { ok: false,
  reason } });
  return alreadyThere ? effects : { ...effects, rejected: [{ opId, op: mine.op,
  reason, quiet: false }] };
}
```

CASE STUDY · FOUND BY REVIEW OR BY A TEST THAT FLAKED

"Would this op change the document?" is not "was this op applied?"

My first rule for `stale` : if the op would still change the document, send it again. A reviewer broke it in six steps: my `gap=1` lands at seq 11 (I never hear); someone sets `gap=2` at seq 12; the server restarts; I resend; `stale` ; the document says 2, so my op "would change it"; I resend with a fresh `baseSeq` ; it lands at seq 13. An old write is now on top of a newer one, and with `add_node` the same path resurrects a node someone deleted. The rule now: only an op that *no earlier connection ever carried* is sent again. The others are given up, and the user is told. A lost edit you know about beats a corrupted document you do not. (The journal in epic 6 remembers every opId for good, and the rule goes away.)

Two smaller lessons from the same code: the dedupe key includes the *sender* (a broadcast shows every opId to every peer), and an acknowledgement is matched by id **and** content: *an id is a claim, the content is the fact.*

6 · A pure core in a thin shell, and dependencies as parameters

Look at what the room asks of the outside world:

FROM `apps/sync/src/room.ts`

```
export type Peer = {
  actor: Actor;
  /** The session this connection was opened with: what presence, role checks and
  revocation look up. */
  session: { userId: string; orgId: string; expiresAt: number };
  send(message: ServerMessage): void;
  /** The same, already serialised. A broadcast is turned into JSON ONCE, not once per
  recipient. */
  sendText?(text: string): void;
  /** What presence shows for this connection; from the verified session. */
  name?: string;
  /** Ends this peer's connection. Called for a peer that keeps sending while it is being
  refused. */
  kick(): void;
  /** E8.2 (F24): may this peer change the document? False for a viewer. The server may
  flip it at any moment (a role change). */
  mayEdit: boolean;
};
```

A peer is "who it is" plus "a way to send to it". No socket. The server builds one from a WebSocket; a unit test builds one from an array; the simulator builds one from a queue. The same goes for time and randomness:

FROM `apps/sync/src/room.ts`

```
rate?: Partial<RateLimit>;
/** The room has no clock of its own: the caller lends it one, and a test lends it a
hand-wound one. */
now?: () => number;
/** How a connection gets its presence id. Injected, like the clock, so that a test can
predict it. */
mintPeerId?: () => string;
/** The room just went read-only. It never retries by itself (no clock): the caller
calls recover() until it answers true. */
onReadOnly?: () => void;
/** An op was applied and announced as `seq`: when the caller counts toward its next
snapshot (E6.2). */
onAccepted?: (seq: number) => void;
};
```

`now` and `mintPeerId` are *functions passed in*, with real defaults. That is the entire dependency-injection mechanism: a parameter with a default value. No container, no interface, no mock library. A test hands in `() => clock.now` and winds the clock by hand.

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
<code>now = Date.now</code> as a default parameter	<code>java.time.Clock</code> injected by Spring	<code>freezegun</code> / a clock argument
<code>type Peer = { send(m): void }:</code> any object with that shape fits	<code>interface Peer</code> , and classes must declare <code>implements</code>	<code>Protocol</code> (structural), or duck typing
a fake = an object literal	Mockito	<code>unittest.mock</code>

Structural typing is what makes this cheap. TypeScript never asks "is this an instance of Peer?"; it asks "does it have `actor`, `session` and `send` of the right types?". In Java terms, every type is an interface and every object implements all the interfaces it happens to fit.

7 • WebSockets: what HTTP did for you, and now you do

- **Authentication happens once**, at the upgrade. A browser cannot set headers on a WebSocket, except one: `Sec-WebSocket-Protocol`. The token rides there (a query string would land in proxy logs). A bad token is an HTTP 401 before any socket exists.
- **There are no status codes** after that, so the server defines close codes that mirror HTTP: 4400, 4404, 4429, 4500, 4503. The client sorts them into "retry" and "never retry".
- **A dead peer says nothing**. The server pings; a peer that has not answered by the next tick is terminated. The browser cannot see pings, so the client has its own watchdog: "I am waiting for something and have heard nothing for 10 s".

- **Backpressure is your job.** `ws.send()` never blocks and never fails: what cannot be written is queued *in your memory*. One stalled reader could grow that without end, so past 1 MB the peer is dropped.
- **Node 24 has the browser's `WebSocket` built in.** That is why one client library serves the browser tab, the AI worker and the git peer.

FROM `apps/sync/src/server.ts`

```

    send: (message) => {
      if (ws.readyState !== ws.OPEN) return;
      // send() never throws and never blocks: what cannot be written yet is queued IN
      OUR MEMORY.
      // A peer that stops reading would grow that queue without end, so past the limit
      it goes.
      if (ws.bufferedAmount > maxBufferedBytes) ws.terminate();
      else ws.send(JSON.stringify(message));
    },
    sendText: (text) => { if (ws.readyState === ws.OPEN && ws.bufferedAmount <=
      maxBufferedBytes) ws.send(text); else if (ws.readyState === ws.OPEN) ws.terminate(); },
    ...(claims.name === undefined ? {} : { name: claims.name }),

```

FROM `packages/peer-client/src/peer.ts`

```

function retryLater(): void {
  if (status === "closed") return;
  readOnly = false; // it belonged to that room; the next welcome says again
  setStatus("offline");
  // The pause starts over only after a connection that LASTED. Resetting it on every
  welcome made
  // "welcome, then drop" (a crash-looping server, a failing database) a reconnect
  every 250 ms, for ever,
  // each one costing the api a session token and the room a copy of the document.
  if (liveSince > 0 && Date.now() - liveSince > retryMs.max) attempt = 0;
  liveSince = 0;
  // Exponential, capped, with jitter: after a server restart a thousand tabs must not
  all return in the same millisecond.
  const delay = Math.min(retryMs.max, retryMs.min * 2 ** attempt++) * (0.5 +
  Math.random() / 2);
  retryTimer = setTimeout(() => void open(), delay);
}

```

CASE STUDY · FOUND BY REVIEW OR BY A TEST THAT FLAKED

Resetting the backoff on every welcome made a reconnect storm.

"Connected again, so start the backoff over" sounds right. But a server that welcomes and then drops you (a crash loop, a failing database) is then retried every 250 ms for ever, by every open tab, and each retry costs the API a session token and the room a copy of the document. The backoff now restarts only after a connection that *lasted*. Jitter is there for the same reason: after a restart, a thousand tabs must not come back in the same millisecond.

8 · Optimistic editing: two documents and one invariant

FROM `packages/peer-client/src/replica.ts`

```
/**
 * One peer's copy of a document. Pure logic: no socket, no timer, no clock, so the
 * reconcile
 * simulator and the unit tests drive this very code (the same split as the room on the
 * server).
 *
 * It holds TWO documents:
 *   confirmed   what the server has said, in the server's order. Never guessed at.
 *   optimistic  confirmed + every op of ours the server has not answered yet. What the
 * user sees.
 * The invariant (tested as a property): optimistic == confirmed, then each pending op,
 * in order.
 */
```

The user must see their edit *now*, not after a round trip. So the replica keeps the server's truth and its own guess apart, and the guess is always *recomputable*: confirmed, then every pending op, in order. When the server says something that changes history (someone else's op landed before mine), the guess is thrown away and made again:

FROM `packages/peer-client/src/replica.ts`

```
/**
 * Throw the guess away and make it again. Only the node MAP is copied; the nodes are
 * shared.
 * That is safe because applyOpInto never edits a node: it puts a NEW node object in
 * the map.
 * Measured on 5,000 nodes with 20 pending: about 1 ms per remote op, against 6.5 ms
 * with structuredClone.
 */
function rebuild(): void {
  revision++;
  optimistic = { ...confirmed, nodes: { ...confirmed.nodes } };
```

```

    for (const each of pending) applyOpInto(optimistic, each.op);
  }

```

That shallow copy is 6x faster than `structuredClone` (about 1 ms against 6.5 ms on 5,000 nodes), and it is only safe because of section 2's promise: applicers replace nodes, they never edit one. The promise is enforced by a test that *freezes* every node the replica holds and then runs sixty random histories; an in-place edit would throw.

The property, stated as a test: after any mix of local edits, remote edits, acknowledgements, refusals and reconnects, `optimistic == confirmed + pending`, the document is well formed, and once nothing is pending the replica's document *is* the server's. Rollback is not a feature that was written; it is what "recompute the guess without the refused op" does.

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
<code>structuredClone(x)</code> : deep copy, built in	serialization or copy constructors	<code>copy.deepcopy</code>
<code>{ ...doc, nodes: { ...doc.nodes } }</code> : one level	<code>new HashMap<>(map)</code>	<code>dict(d)</code>
getters on a returned object (<code>get doc()</code>) expose live state read-only	getter methods	<code>@property</code>

9 • Rate limiting: where you charge matters more than how

FROM `apps/sync/src/room.ts`

```

/** Takes one token (unless `take` is false), or says how many ms until there is one. 0
= taken. */
function spend(actor: Actor, take = true): number {
  const key = bucketKey(actor);
  const bucket = buckets.get(key) ?? { tokens: rate.burst, at: now() };
  // max(0): a clock that was corrected BACKWARDS must not turn into a debt of that
  many milliseconds.
  bucket.tokens = Math.min(rate.burst, bucket.tokens + (Math.max(0, now() - bucket.at)
/ 1000) * rate.perSecond);
  bucket.at = now();
  buckets.set(key, bucket);
  if (bucket.tokens >= 1) {
    if (take) bucket.tokens -= 1;
    return 0;
  }
  // "Come back when it is worth it": when a few tokens have gathered, not the first
  one. A client told
  // to return for ONE token sends one op, is refused on the next, and pays a refusal
  for every op.
  // Capped: a budget of 0 per second would make this Infinity, which JSON writes as

```

```

null.
    const worthIt = Math.min(rate.burst, WORTH_RETURNING_FOR);
    return Math.min(MAX_RETRY_AFTER_MS, Math.ceil(((worthIt - bucket.tokens) /
rate.perSecond) * 1000));
}

```

A token bucket is ten lines. The decisions around it are the design:

- **Charge on arrival.** The first version charged when the queue reached the op. Behind a slow journal the bucket refills while ops wait: a flood of 3,000 frames got *zero* refusals and sat in memory. *Limit where work arrives, not where it is processed.*
- **Per actor, not per connection:** otherwise reconnecting is a free refill. And an AI run has its own bucket, so it cannot starve the person who started it.
- **Keep the order.** After op K is refused, the bucket refills, and op K+1 (K's child) would get in and be lost as `gone`. So the room refuses that peer's later ops until K comes back.
- **A strike is coming back too soon,** not being refused. A second tab that waits as told and loses the race for the token is unlucky, not abusive, and must never be dropped.
- **Cheap requests must not share a pool with expensive ones.** Ops that change nothing cost almost nothing; remembered in the same memory as real ops, 20,000 of them pushed out every real entry and turned honest resends into `stale`.

CASE STUDY · FOUND BY REVIEW OR BY A TEST THAT FLAKED

My own honest client cost 18,524 refusals for 600 ops.

Told "come back in 10 ms", it came back with its whole window of 50 ops. One token had refilled. One op in, 49 refused, repeat. The test was named "an honest client is never dropped" and it passed, because it never *counted* refusals. Now the window restarts at one op and grows by one per answer (the shape of TCP's congestion control), the server's hint means "when a few tokens are back", and the test counts. *Assert the thing you claim, in numbers.*

10 · Presence: the message that is not an op

Cursors and selections are not part of the document: never ordered, never stored, gone with the process. So presence takes none of the op pipeline: no queue, no seq, no persist, no budget. What it needs instead:

- **Identity from the token,** never from the message. The client message is a `strictObject`: an extra `name` field closes the connection.
- **Latest wins, nothing queues.** The client sends at most every 50 ms and always the newest state; the room drops anything faster than 25 ms apart. 10,000 frames in one tick relay 2.

- **A dead connection says no goodbye**, and the server may not notice for 15 s. So every client refreshes every 2 s, and *viewers* forget anyone silent for 5 s.
- **A cursor is a fraction of the canvas**, not pixels: two windows are never the same size.
- **Serialise once**. N peers at 20 messages a second is $N^2 \cdot 20$ `JSON.stringify` calls if you do it per recipient.

FROM `apps/sync/src/room.ts`

```
/** To everyone (but `except`). Serialised once: with N peers at 20 presence messages a
second each, once per recipient is N x N x 20 stringifies. */
function broadcast(message: ServerMessage, except?: Peer): void {
  let text: string | undefined;
  for (const each of peers) {
    if (each === except) continue;
    if (each.sendText) each.sendText((text ??= JSON.stringify(message)));
    else each.send(message);
  }
}
```

11 · React meets a store that is not React's

You know servers; here is the smallest honest account of the browser side. React re-renders a component when *its* state changes. The peer is not React state: it changes when a WebSocket frame arrives. `useSyncExternalStore` is the bridge: "here is how to subscribe, here is how to read a snapshot". React re-renders when the snapshot is a *different value* (`Object.is`).

FROM `apps/web/src/usePeer.ts`

```
function openStore(documentId: string, onRejected: (rejection: Rejection) => void, onOp:
(message: SequencedOp) => void, onSession: (session: SessionResponse) => void) {
  const listeners = new Set<() => void>();
  const tell = (): void => { for (const listener of listeners) listener(); };
  // Every session minted passes here (a reconnect too), so the role the editor shows
  controls by is the api's latest word.
  const session = async (): Promise<SessionResponse | null> => {
    const minted = await openSession(documentId);
    if (minted) onSession(minted);
    return minted;
  };
  const peer = connectPeer({ manifest, session, onChange: tell, onStatus: tell,
onRejected, onOp });
  return {
    peer,
    subscribe: (listener: () => void) => { listeners.add(listener); return () => {
listeners.delete(listener); }; },
    // peer.doc is edited IN PLACE and never changes identity, so the snapshot is built
    from what does move.
    snapshot: () =>
```

```
`${String(peer.revision)}:${String(peer.pendingCount)}:${peer.status}:${String(peer.readOnly)}:${String(peer.presenceRevision)}`,
  };
}
```

The document is edited in place, so its identity never changes and would make a useless snapshot. The snapshot is a string of the numbers that do move.

FROM `apps/web/src/usePeer.ts`

```
export function usePeer(documentId: string, onOp?: (message: SequencedOp) => void) {
  const [refusals, setRefusals] = useState<Refusal[]>([]);
  const [store, setStore] = useState<ReturnType<typeof openStore>>();
  const [role, setRole] = useState<Role>();
  const refuse = (reason: Reason, op?: Op): void => { setRefusals((before) => [...before,
  { id: crypto.randomUUID(), reason, ...(op === undefined ? {} : { op }) }]); };
  const latestOnOp = useRef(onOp);
  latestOnOp.current = onOp;
  useEffect(() => {
    // 'quiet' (someone else removed the node first) is not news: the canvas already
    shows it (F5).
    const opened = openStore(documentId, (rejection) => { if (!rejection.quiet)
    refuse(rejection.reason, rejection.op); }, (message) => latestOnOp.current?.(message),
    (session) => { setRole(session.role); });
    setStore(opened);
    return () => { opened.peer.close(); };
  }, [documentId]);
  useSyncExternalStore((store ?? NOTHING).subscribe, (store ?? NOTHING).snapshot);
  return { peer: store?.peer, role, refusals, refuse, dismiss: (id: string): void => {
  setRefusals((before) => before.filter((each) => each.id !== id)); }; }
}
```

- **Side effects live in effects.** A render must be pure: React may run it twice and discard one (StrictMode does so on purpose in development). An effect comes with a cleanup; a render does not.
- **Derived, not stored.** The selection is `doc.nodes[wanted] ?? root`. When someone else deletes the selected node there is no state to repair.
- **A ref for what the screen does not depend on.** The pointer position changes sixty times a second and nothing on *our* screen shows it: `useRef`, not `useState`.
- **key resets a component.** The inspector is keyed by node id: select another node and the old form unmounts (its field blurs, its draft is committed to the *right* node) and a fresh one mounts.

CASE STUDY · FOUND BY REVIEW OR BY A TEST THAT FLAKED

StrictMode left two live connections per page.

Mount, cleanup, mount again, all in one tick. My library *scheduled* its connection (`queueMicrotask`), so the cleanup's `close()` ran before the connection had started, and the start then undid the close. A library that defers its start must ask "was I closed meanwhile?" when it finally runs.

REACT	CLOSEST JAVA IDEA	CLOSEST PYTHON IDEA
<code>useState</code> : state owned by the component	a field + <code>repaint()</code>	n/a
<code>useEffect(() => { open(); return close; }, [id])</code>	<code>@PostConstruct</code> / <code>@PreDestroy</code>	a context manager
the prop form is generated from the manifest at runtime	reflection over a DTO	pydantic model fields

12 · Types as data: reading TypeScript with TypeScript

The canvas must know which components exist and which props they take. Nobody wrote that list: it is *extracted* from the sample app's own TypeScript types with the compiler API, and committed. Change a prop in the design system and `make check` fails until the manifest is regenerated.

The compiler is a library: `ts.createProgram` gives you a type checker you can ask "what are the properties of this type, and is this one optional?". Java's equivalent is an annotation processor or reflection; the difference is that TypeScript's types do not exist at runtime, so the only way to get them into data is to run the compiler.

CASE STUDY · FOUND BY REVIEW OR BY A TEST THAT FLAKED

Arrow-function components were silently skipped.

The extractor looked for `function Button(props: ...)` declarations. `export const Button = (props: ...) => ...` is a *variable* whose initialiser is a function: a different syntax node. Half of all React code is written that way. Code that walks a syntax tree needs a test per way of writing the same thing.

13 · How you test concurrency without sleeping

- **Property tests:** not "this input gives that output" but "for every history, this stays true". Seeded, so a failure replays.
- **A simulator** (apps/sync/src/sim.ts, `make sim`): the real room and real replicas, a network that delays, reorders and drops, a server that restarts, one seeded generator deciding everything. Same seed, same run, byte for byte, in separate processes.
- **Mutation:** break a rule on purpose and demand a red test. The simulator's first version passed when a replica re-applied a late acknowledgement, because random ops almost never made two peers write the same property. It now generates *collisions* on purpose and checks every caught-up replica at every step, not only at the end (later edits hide a wrong value).
- **Held frames:** in the browser tests, Playwright sits between the page and the server and holds one browser's outgoing frames. "Two people did this at the same moment" becomes a deterministic test.
- **Injected time:** a hand-wound clock instead of `sleep`.

FROM apps/sync/src/sim.ts

```
function check(): void {
  for (const problem of checkDoc(room.doc)) fail(`the room's document is malformed:
  ${problem}`);
  for (const client of clients) for (const problem of checkDoc(client.replica.doc))
  fail(`${client.name}'s document is malformed: ${problem}`);
  // A replica that has heard everything and waits for nothing must hold the room's
  document NOW, not
  // only at the end: later edits overwrite a wrong value and would hide the divergence
  by then.
  const roomNow = canon(room.doc);
  for (const client of clients) {
    const caughtUp = client.peer !== undefined && client.toClient.length === 0 &&
    client.replica.pendingCount === 0 && client.replica.seq === room.seq;
    if (caughtUp && canon(client.replica.doc) !== roomNow) fail(`${client.name} has
    heard everything up to seq ${String(room.seq)} and still holds a different document`);
  }
  for (const node of Object.values(room.doc.nodes)) {
    const expected = canon(Object.fromEntries(lastWritten.get(node.id) ?? []));
    if (node.parentId !== null && canon(node.props) !== expected) fail(`last writer did
    not win on ${node.id}: the room holds ${canon(node.props)}, the latest writes say
    ${expected}`);
  }
  // The rest of the rules (a remove beats an edit, index = final position) in one
  check: the room's
  // document is exactly what applying the sequenced ops in seq order produces,
  recomputed with the
  // PURE applier (the room uses the in-place one): it catches a room that skips,
  repeats or reorders.
  const model = [...log].sort((a, b) => a.seq - b.seq).reduce<Doc>((doc, each) =>
  applyOp(doc, each.op), emptyDoc());
```

```
if (canon(model) !== canon(room.doc)) fail("the room's document is not what its own
sequence of ops produces");
}
```

CASE STUDY · FOUND BY REVIEW OR BY A TEST THAT FLAKED

The simulator's best find was a bug in code that every committed seed passed.

Two features, each correct alone. The room remembers "this op changed nothing" and answers a resend with the same `ack` (re-judging it later could put an old value over a newer one). The replica, told never to trust blindly, refused to believe an `ack` for an op that *would* change the document it holds now. Put together: I set `gap=2` when it is already 2; the `ack` is lost; someone sets `gap=1`; I reconnect and resend; the room says `ack`; I do not believe it, reconnect, resend; for ever, silently. It happens in about one seed in 700, and seeds 1 to 40 never hit it. A verifier swept 3,000 seeds and seed 761 printed the loop. The fix is one deleted condition: the room's verdict is about *then*, not now, so the `ack` is believed and the newer value stands.

Three habits came out of it: after building a simulator, **sweep thousands of seeds once** and commit the ones that fail; make the final drain **bounded**, so a livelock is a failure with a trace and not a hang; and when a client double-checks a server, it needs the server's real semantics, or the check itself becomes the bug.

The independent last-writer-wins check exists because the "replay the log" check uses doc-model's own applier: a wrong rule inside it would agree with itself. **An oracle must not share code with what it judges.**

14 · What the type system could not see, epic 2 edition

Everything below compiled under `strict`, passed the linter and the first tests.

WHAT WENT WRONG	WHY	FIX
frame lost while the room was loading	listener attached after an <code>await</code>	attach first, buffer, replay
shutdown lost unsaved edits	close handlers run on a later tick	save first, then close
resend applied twice outside the dedupe window	bounded memory with no rule for the outside	<code>baseSeq</code> + <code>stale</code>
old write over a newer one after a restart	"would change" mistaken for "was not applied"	only never-sent ops are rebased
a co-editor could diverge your canvas for good	ack matched by id alone	id AND content
a cyclic welcome froze the tab	contract checks nodes, not the tree	<code>checkDoc</code> on arrival; seen-set in the walk
~6 reconnects a second, for ever	backoff reset on every welcome	reset after a connection that lasted
an honest second tab was kicked	a strike per refusal	a strike per EARLY return
floods never refused behind a slow journal	budget charged inside the queue	charge on arrival
honest resends turned <code>stale</code>	no-ops shared the dedupe memory	their own small memory
18,524 refusals for 600 ops	whole window resent into one token	restart at 1, +1 per answer
two live connections per page in dev	<code>close()</code> before a deferred start was undone	check "closed?" at the start
a stale link retried for ever	404 mapped to "retry later"	throw = retry, null = give up
<code>1e999</code> silently deleted a prop	a number input reports "" for unreadable text	<code>validity.badInput</code>
a third of presence colours unreadable	a colour formula cannot promise contrast	a checked list + a test that recomputes WCAG

an endless, silent resync loop

the replica disbelieved a
REMEMBERED ack

the ack is about then, not now:
believe it

`__proto__` as a node id

plain objects inherit

refuse in the contract;
`Object.hasOwnProperty` everywhere

The pattern: types prove shapes. They say nothing about *order, time, repetition, memory* or *who is speaking*, and a distributed system is made of exactly those. That is what the verifier pass, the property tests, the simulator and mutation-checking are for.

Your turn: three drills

Do these alone. They live in `drills/lesson-2/` and both coded drills start RED; `sh drills/lesson-2/check.sh` shows your progress.

D1 · Extend the protocol, end to end. Presence should also say whether a peer is *typing* in a prop field: add `typing: boolean`. You will touch the contract (both directions: the client message is strict, the server message is loose), the room (relay it, and remember it for newcomers) and, if you like, the client and the canvas. The drill's test checks the contract and the room. `make check` must stay green: the gate applies to you too.

Predict first: which existing test will fail the moment you add a required field to `Presence`, and what does that tell you about required versus optional fields on a wire that old clients share? (Section 1's case study.)

D2 · Fix a planted bug. `drills/lesson-2/d2-replica/replica.ts` is a 60-line replica with one bug from this chapter planted in it. Its test plays a history in which two people set the same property around a reconnect, and expects the replica to end with the server's value. Form a theory from the failing assertion before you read the code.

D3 · Explain a decision before reading mine. Answer out loud or on paper, then open the answer.

▼ 1. Why is the server the only thing that orders ops, instead of a CRDT that lets every peer merge on its own?

A design tree has invariants a merge function cannot see locally: no cycles, a parent must accept children, props must match the component. A single sequencer makes every peer apply the same ops in the same order through the same validator, so the

invariants hold by construction, and "who won" is one integer comparison. The price is that editing needs the server: offline edits are optimistic guesses that may be refused. For a team tool with a server anyway (auth, AI, git), that is the cheap side of the trade. A CRDT earns its complexity when peers must merge with no server at all.

▼ 2. The replica is pure (no socket, no timer) and the transport is separate. What did that buy, concretely?

Seventy-odd rules tested in under a second with no network; a property test over random histories; a simulator that drives the real code deterministically; and one client for the browser, the AI worker and the git peer, because only the thin shell knows what a WebSocket is. The same split on the server (room versus server) bought the same things.

▼ 3. Presence is relayed by the same room. Why does it not go through the op pipeline?

It has none of an op's needs and all of its costs. Nobody needs cursors in order, durable, deduplicated or replayable; they need the latest one, now. Through the pipeline every pointer move would take a sequence number, a journal row and a slot in the dedupe memory, and a fast pointer would starve real edits of their budget. So: no queue, no seq, no persist, latest wins, and silence removes a peer.

Recall first

Look away. Name the room's checks in order, from a frame arriving to a broadcast, and for two of them say which bug put them where they are. Then check.

▼ Check yourself

Budget, on arrival (floods piled up behind a slow journal) → seen this (sender, opId)? answer as before (honest resends were refused as duplicates) → older than anything remembered? **stale** (a resend was applied twice) → the document's rules, precise reason → the room's limits (a move escaped the depth cap) → persist, and only then take the seq (no gaps, nobody hears of an op that could be lost) → apply → remember → broadcast, serialised once; the sender's copy is its acknowledgement.

Then test it

1. In the `default` branch of a switch over all four op types, what is the type of `op`, and why assign it to a `never` variable?

Op; it documents the fallback

any; it silences the linter

`never`; a fifth op type then stops the build

unknown; it forces a cast

After every member is handled nothing is left: the empty type. The assignment only compiles while that stays true.

2. The room has no lock. Why can two ops still interleave wrongly without the promise queue?

Node runs handlers on a thread pool

`await persist()` is a point where other code runs

Garbage collection can pause a handler

WebSocket frames are parsed in parallel

Single-threaded means interleaving only at an `await`. The queue makes each op wait for the one before it.

3. Why does the room check "have I seen this opId?" BEFORE validating the op?

To save a sequence number

An honest resend of add_node would be refused as duplicate_node

Validation is slower than a map lookup

Because the contract requires it

A client that lost its acknowledgement sends again and must get the ORIGINAL answer.

4. An op comes back `stale` after a server restart. It was already on the wire of an earlier connection. What does the replica do, and why?

Gives it up and tells the user: it may have been applied and then overwritten

Reloads the page

Resends it with a fresh baseSeq

Resends it only if it would still change the document

"Would change the document" is not "was not applied". Resending could put an old write over a newer one, or re-add a deleted node.

5. `rebuild()` copies only the node MAP and shares the node objects with the confirmed document. What makes that safe?

JavaScript objects are copied on assignment

The confirmed document is never read again

`Object.freeze` is called on every node in production

Apppliers never edit a node; they put a new node object in the map

Immutability by discipline, enforced by a property test that freezes the nodes and runs sixty random histories.

6. Where is the rate-limit token spent, and what went wrong when it was spent elsewhere?

In `handle()`, after validation, to avoid charging for bad ops

On arrival, in `submit()`; inside the queue a slow journal let the bucket refill, so floods were never refused

In the WebSocket server, per frame, to save the room's time

In the client, to avoid a round trip

Limit where work ARRIVES, not where it is processed.

7. Why is `peer.doc` useless as a `useSyncExternalStore` snapshot?

It changes on every render and causes a loop

It is edited in place, so its identity never changes and React sees no difference

React cannot hold objects in a snapshot

It is too large to compare

React compares snapshots with `Object.is`. The snapshot is built from `revision`, `pendingCount`, `status` and `presenceRevision`.

8. The simulator's replay check passed even when last-writer-wins was broken inside doc-model. Why?

The check used doc-model's own applier, so the wrong rule agreed with itself

The seeds were too few

Set-prop ops were never generated

The room caught it first

An oracle must not share code with what it judges. The fix computes "the latest write per prop" from first principles.

Next: Lesson 3, after epic 3. An AI agent joins the document as one more peer: the Claude Agent SDK, tools that emit ops, streaming, cancellation, and what "the AI did this" must mean in an audit log.